

박수민\_0618\_세탁기구현

**#main.c**

#define F\_CPU 16000000UL

#include <avr/io.h>

#include <avr/interrupt.h> // sei, ISR 등

#include <stdio.h>

#include "button.h"

extern void init\_button(void);

extern void init\_led();

extern int get\_button(int button\_num, int button\_pin);

extern int init\_fnd();

extern void init\_uart0();

extern void UART0\_transmit(uint8\_t data);

extern void pc\_command\_processing();

extern void init\_ultrasonic(void);

extern void make\_trigger(void);

extern void ultrasonic\_processing(void);

extern void init\_timer3\_pwm(void);

extern void init\_motor\_driver(void);

extern void dcmotor\_pwm\_control\_main(void);

extern void init\_washer(void);

extern void washer\_run(void);

extern void init\_timer2(void);

extern volatile char scm[50];

FILE OUTPUT = FDEV\_SETUP\_STREAM(UART0\_transmit, NULL, \_FDEV\_SETUP\_WRITE);

void init\_timer0();

volatile uint32\_t msec\_count = 0; // volatile 의 최적화 방지

volatile int ultrasonic\_check\_time = 0;

ISR(TIMER0\_OVF\_vect)

{

TCNT0 = 6; // TCNT0 6~256: 250 펄스 count 하기 위해

msec\_count++;

ultrasonic\_check\_time++;

}

```

int main(void)
{
    init_led();
    init_timer0();
    init_uart0();
    init_button();
    init_washer();
    init_fnd();
    init_timer2();
    init_timer3_pwm();
    init_motor_driver();

    stdout = &OUTPUT;          // printf가 동작할 수 있도록 stdout을 설정
    sei();                      // 전역(대문) interrupt 허용

    // dcmotor_pwm_control_main();

    while (1)
    {
        washer_run();
        // pc_command_processing();
        // ultrasonic_processing();
    }
}

void init_timer0()
{
    // TCNT0 = TimerCount0
    // TCNT0 6 ~ 256 : 250개 펄스를 count 하기 위해 6부터 시작
    TCNT0 = 6;

    TCCR0 &= ~( 1<< CS02 | 1<< CS01 | 1<< CS00); // 0분주 reset 먼저
    TCCR0 |= 1<< CS02 | 0<< CS01 | 0<< CS00;      // 64분주
    TIMSK |= 1<< TOIE0;    // TIMER0 Overflow INT
}

```

```

#fnd.c
/*
 * fnd.c
 *
 * Created: 2026-06-12 오후 1:22:04
 * Author: kccistc
 */

#include "fnd.h"
#include "button.h"
#include <avr/interrupt.h>

/* ——— 모드 정의 ——— */
#define MODE_CLOCK    0    // MM.SS 시계 (도트 1초마다 깜빡임)
#define MODE_COUNT    1    // 초시계      (오른쪽 초 카운트 + 왼쪽 원형 애니메이션)
#define MODE_SW       2    // 스톱워치   (버튼1로 start/stop)

#define ANIM_PERIOD    100 // 원형 애니메이션 한 칸 이동 주기(ms)

volatile uint32_t ms_count    = 0;    // 시계용 ms
volatile uint32_t sec_count   = 0;    // 시계용 sec
volatile uint8_t  dot_display = 0;    // 시계 가운데 도트
volatile uint32_t cnt_ms      = 0;    // 초시계용 ms
volatile uint32_t sw_ms       = 0;    // 스톱워치 누적 ms
volatile uint8_t  anim_step   = 0;    // 초시계 원형 애니메이션 위치 (0~7)

extern volatile uint32_t msec_count;    // Timer2 1ms 카운터 (최적화 방지)

extern int get_button(int button_num, int button_pin);

int fnd_main();
void init_fnd();
void init_timer2();
void fnd_display();
void fnd_display_clock(uint32_t total_sec, uint8_t dot_on);
void fnd_display_stopwatch(uint32_t total_ms);
void fnd_display_count(uint32_t total_sec, uint8_t step);
void fnd_all_off();

void fnd_display_washer(uint16_t remain_time, uint16_t remain_msec);
void fnd_display_setting(uint16_t value);

```

```

static const uint8_t fnd_font[] = {
    0xC0, 0xF9, 0xA4, 0xB0, 0x99, /* 0 1 2 3 4 */
    0x92, 0x82, 0xD8, 0x80, 0x98, /* 5 6 7 8 9 */
    0x40, 0x79, 0x24, 0x30, 0x19, /* 0. 1. 2. 3. 4. */
    0x12, 0x02, 0x58, 0x00, 0x18 /* 5. 6. 7. 8. 9. */
};

ISR(TIMER2_OVF_vect)
{
    TCNT2 = 194; /* 62tick 뒤 다시 overflow 되도록 reload
    // msec_count++;
}

void init_timer2(void)
{
    TCNT2 = 194; /* 1ms 기준 reload 값
    TCCR2 &= ~(1 << CS22) | (1 << CS21) | (1 << CS20); // 분주 비트 reset
    TCCR2 |= (1 << CS22); // 256분주 (ATmega128:
    CS22:20 = 100)
    TIMSK |= (1 << TOIE2); // Timer2 overflow 인터럽트 허
용
    sei(); // 전역 인터럽트 허용
}

int fnd_main(void)
{
    uint8_t mode = MODE_CLOCK;
    uint8_t sw_run = 0; // 스톱워치 동작 여부
    uint32_t last_ms = 0; // 마지막으로 처리한 msec_count

    ms_count = 0; sec_count = 0; dot_display = 0;
    cnt_ms = 0; sw_ms = 0; anim_step = 0;

    while (1)
    {
        // 1. 화면 갱신 : 매 루프마다 한 자리씩 멀티플렉싱 */
        switch (mode)
        {
            case MODE_CLOCK: fnd_display_clock(sec_count, dot_display);
break;

            case MODE_COUNT: fnd_display_count(cnt_ms / 1000,
anim_step); break;

            case MODE_SW: fnd_display_stopwatch(sw_ms);

```

break;

}

/\* 2. 1ms tick 이 지났을 때만 시간 진행 + 버튼 처리 \*/

if (msec\_count != last\_ms)

{

    last\_ms = msec\_count;

    ms\_count++;

    if (ms\_count >= 1000) { ms\_count = 0; sec\_count++; }

    if (ms\_count == 500) { dot\_display = !dot\_display; }

    if (mode == MODE\_COUNT)

    {

        cnt\_ms++;

        if (cnt\_ms % ANIM\_PERIOD == 0)

        anim\_step = (anim\_step + 1) % 8;

    }

    if (mode == MODE\_SW && sw\_run) sw\_ms++;

/\* 버튼0 : 모드 전환 \*/

if (get\_button(BUTTON0, BUTTON0PIN))

{

    switch (mode)

    {

        case MODE\_CLOCK: mode = MODE\_COUNT;

cnt\_ms = 0; break;

        case MODE\_COUNT: mode = MODE\_SW;

sw\_ms = 0; sw\_run = 0; break;

        case MODE\_SW: mode = MODE\_CLOCK;

break;

    }

}

/\* 버튼1 : 스톱워치 모드에서만 start/stop 토글 \*/

if (get\_button(BUTTON1, BUTTON1PIN))

{

    if (mode == MODE\_SW)

    sw\_run = !sw\_run;

}

/\* 버튼2 : 리셋 등 \*/

```

        if (get_button(BUTTON2, BUTTON2PIN))
        {
            if (mode == MODE_SW)
            {
                if (sw_run == 1)          { sw_ms = 0; sw_run
= 0; } // 달리는 중 -> 리셋+정지

                else if (sw_ms == 0)      { sw_run = 1; }

                // 리셋 상태 -> 시작

                else                      { sw_ms = 0; sw_run
= 0; } // 정지 상태 -> 리셋+정지
            }
            else                          // 다른 모드 -> 전체 리셋
            {
                ms_count = 0; sec_count = 0; dot_display = 0;
                cnt_ms = 0;  anim_step = 0;
                sw_ms = 0;   sw_run = 0;
            }
        }
    }

    return 0;
}

```

```

void init_fnd(void)
{
    FND_DATA_DDR = 0xff; // 출력모드로 설정한다.
    FND_DIGIT_DDR |= 1 << FND_DIGIT_D1 | 1 << FND_DIGIT_D2 | 1 <<
FND_DIGIT_D3 | 1 << FND_DIGIT_D4;

    // FND를 전체 off 하는 작업
    #if 1          // common anode 방식

        FND_DATA_PORT = 0xff;

    #else          // common cathode 방식

        FND_DATA_PORT ^= 0xff;

    #endif

    FND_DIGIT_PORT
&=

```

```
~((1<<FND_DIGIT_D1)|(1<<FND_DIGIT_D2)|(1<<FND_DIGIT_D3)|(1<<FND_DIGIT_D4));
```

```
}
```

```
void fnd_all_off(void)
```

```
{
```

```
    FND_DIGIT_PORT &= ~((1 << FND_DIGIT_D1) | (1 << FND_DIGIT_D2) | (1 << FND_DIGIT_D3) | (1 << FND_DIGIT_D4));
```

```
    FND_DATA_PORT = 0xFF;
```

```
}
```

```
void fnd_display_washer(uint16_t remain_time, uint16_t remain_msec)
```

```
{
```

```
    static uint8_t digit = 0;
```

```
    FND_DIGIT_PORT = 0x00;
```

```
    if(remain_time >= 60)
```

```
    {
```

```
        // MM:SS 표시
```

```
        uint8_t sec1 = remain_time % 10;
```

```
        uint8_t sec10 = (remain_time / 10) % 6;
```

```
        uint8_t min1 = (remain_time / 60) % 10;
```

```
        uint8_t min10 = (remain_time / 600) % 10;
```

```
        switch(digit)
```

```
        {
```

```
            case 0:
```

```
                FND_DATA_PORT = fnd_font[sec1];
```

```
                FND_DIGIT_PORT = 0x80;
```

```
                break;
```

```
            case 1:
```

```
                FND_DATA_PORT = fnd_font[sec10];
```

```
                FND_DIGIT_PORT = 0x40;
```

```
                break;
```

```
            case 2:
```

```

        FND_DATA_PORT = fnd_font[min1 + 10]; // dot
        FND_DIGIT_PORT = 0x20;
        break;

    case 3:
        FND_DATA_PORT = fnd_font[min10];
        FND_DIGIT_PORT = 0x10;
        break;
    }
}
else
{
    // SS.CC 표시
    uint16_t cs = remain_time * 100 + remain_msec / 10;

    uint8_t cs1 = cs % 10;
    uint8_t cs10 = (cs / 10) % 10;
    uint8_t sec1 = (cs / 100) % 10;
    uint8_t sec10 = (cs / 1000) % 10;

    switch(digit)
    {
        case 0:
            FND_DATA_PORT = fnd_font[cs1];
            FND_DIGIT_PORT = 0x80;
            break;

        case 1:
            FND_DATA_PORT = fnd_font[cs10];
            FND_DIGIT_PORT = 0x40;
            break;

        case 2:
            FND_DATA_PORT = fnd_font[sec1 + 10]; // dot
            FND_DIGIT_PORT = 0x20;
            break;

        case 3:
            FND_DATA_PORT = fnd_font[sec10];
            FND_DIGIT_PORT = 0x10;
            break;
    }
}

```



```

        digit = (digit + 1) % 4;
    }

    void fnd_display_setting(uint16_t value)
    {
        static uint8_t digit = 0;

        uint8_t one = value % 10;
        uint8_t ten = (value / 10) % 10;

        FND_DIGIT_PORT = 0x00;

        switch(digit)
        {
            case 0:    // 가장 오른쪽
                FND_DATA_PORT = fnd_font[one];
                FND_DIGIT_PORT = 0x80;
                break;

            case 1:
                FND_DATA_PORT = fnd_font[ten];
                FND_DIGIT_PORT = 0x40;
                break;

            case 2:
                FND_DATA_PORT = 0xFF;
                FND_DIGIT_PORT = 0x20;
                break;

            case 3:
                FND_DATA_PORT = 0xFF;
                FND_DIGIT_PORT = 0x10;
                break;
        }

        digit = (digit + 1) % 4;
    }
}

```

#washer.c

/\*

\* washer.c

\*

\* Created: 2026-06-18 오후 3:58:53

\* Author: kccistc

\*/

#define F\_CPU 16000000UL

#include <avr/io.h>

#include <avr/interrupt.h>

#include <stdio.h>

#include "washer.h"

#include "button.h"

#include "pwm.h"

#include "fnd.h"

#include "led.h"

uint16\_t wash\_time = 2; // 세탁 시간

uint16\_t rinse\_time = 1; // 헹굼 시간

uint16\_t spin\_time = 1; // 탈수 시간

uint16\_t remain\_time = 0;

uint16\_t remain\_msec = 0;

uint32\_t prev\_ms = 0;

extern void init\_button(void);

extern int get\_button(int button\_num, int button\_pin);

extern void fnd\_display\_washer(uint16\_t remain\_time, uint16\_t remain\_msec);

extern void fnd\_display\_setting(uint16\_t value);

extern volatile uint32\_t msec\_count;

static uint32\_t motor\_dir\_time = 0;

static uint8\_t motor\_dir = 0;

static WasherState state = WASHER\_IDLE;

static WasherState prev\_state = WASHER\_IDLE;

void init\_washer(void);

void washer\_run(void);

static void washer\_timer\_update(void);

```
static void washer_button_process(void);
static void washer_state_process(void);
static void washer_motor_control(void);
```

```
char *state_string(void)
{
    switch(state)
    {
        case WASHER_IDLE:
            return "IDLE";

        case SET_WASH_TIME:
            return "SET_WASH";

        case SET_RINSE_TIME:
            return "SET_RINSE";

        case SET_SPIN_TIME:
            return "SET_SPIN";

        case STATE_WASH:
            return "WASH";

        case STATE_RINSE:
            return "RINSE";

        case STATE_SPIN:
            return "SPIN";

        case STATE_PAUSE:
            return "PAUSE";
    }

    return "UNKNOWN";
}
```

```
void init_washer(void) {

    state = WASHER_IDLE;

    wash_time = 1;
```

```

    rinse_time = 1;
    spin_time = 1;
    motor_dir_time = msec_count; // ← 이거 추가
    motor_dir = 0;
    remain_time = 0;

}

```

```

static void washer_timer_update(void)
{
    if(msec_count != prev_ms)
    {
        prev_ms = msec_count;

        if(state == STATE_WASH ||
           state == STATE_RINSE ||
           state == STATE_SPIN)
        {
            if(remain_time > 0 || remain_msec > 0)
            {
                if(remain_msec == 0)
                {
                    if(remain_time > 0)
                    {
                        remain_time--;
                        remain_msec = 999;
                    }
                }
                else
                {
                    remain_msec--;
                }
            }
        }
    }
}

```

```

static void washer_button_process(void)
{
    // BTN1

```

```

if(get_button(BUTTON1, BUTTON1PIN))
{
    switch(state)
    {
        case WASHER_IDLE:
            state = SET_WASH_TIME;
            break;

        case SET_WASH_TIME:
            state = SET_RINSE_TIME;
            break;

        case SET_RINSE_TIME:
            state = SET_SPIN_TIME;
            break;

        case SET_SPIN_TIME:
            state = WASHER_IDLE;
            break;

        default:
            break;
    }
}

```

```

// BTN2

```

```

if(get_button(BUTTON2, BUTTON2PIN))
{
    switch(state)
    {
        case SET_WASH_TIME:
            wash_time++;
            if(wash_time > 9)
                wash_time = 1;
            break;

        case SET_RINSE_TIME:
            rinse_time++;
            if(rinse_time > 9)
                rinse_time = 1;
            break;
    }
}

```

```

        case SET_SPIN_TIME:
            spin_time++;

            if(spin_time > 9)
                spin_time = 1;

            break;

        default:
            break;
    }
}

// BTN0 시작 / 정지

if(get_button(BUTTON0, BUTTON0PIN))
{
    switch(state)
    {
        case WASHER_IDLE:

            state = STATE_WASH;
            remain_time = wash_time * 10;
            remain_msec = 0;
            motor_dir_time = msec_count;
            printf("START\r\n");

            break;

            case STATE_WASH:
            case STATE_RINSE:
            case STATE_SPIN:

            prev_state = state;
            state = STATE_PAUSE;

            printf("PAUSE\r\n");

            break;

            case STATE_PAUSE:

            state = prev_state;

```

```

        printf("RESUME\r\n");

        break;

    default:
        break;
    }
}

if(get_button(BUTTON3, BUTTON3PIN))
{
    if(state == STATE_PAUSE)
    {
        state = WASHER_IDLE;

        remain_time = 0;
        remain_msec = 0;

        OCR3C = 0;

        PORTF &= ~((1<<6)|(1<<5));

        printf("CANCEL\r\n");
    }
}

static void washer_state_process(void)
{
    switch(state)
    {
        case STATE_WASH:

            if(remain_time == 0 && remain_msec == 0)
            {
                state = STATE_RINSE;
                remain_time = rinse_time * 10;
                remain_msec = 0;
            }

            break;

        case STATE_RINSE:

```

```

        if(remain_time == 0 && remain_msec == 0)
        {
            state = STATE_SPIN;
            remain_time = spin_time * 10;
            remain_msec = 0;
        }

        break;

    case STATE_SPIN:

        if(remain_time == 0 && remain_msec == 0)
        {
            state = WASHER_IDLE;
        }

        break;

    default:
        break;
    }
}

```

```

static void washer_motor_control(void)
{
    switch(state)
    {
        case WASHER_IDLE:

            OCR3C = 0;

            PORTF &= ~((1<<6)|(1<<5));

            break;

        case STATE_PAUSE:

            OCR3C = 0;

            PORTF &= ~((1<<6)|(1<<5));

```



```

        break;

case STATE_WASH:
{
    OCR3C = 120;

    uint32_t cycle_time = (msec_count - motor_dir_time) % 8000;

    if(cycle_time < 3000)
    {
        // 정회전
        PORTF &= ~((1<<6)|(1<<5));
        PORTF |= (1<<6);
    }
    else if(cycle_time < 4000)
    {
        // 정지 1초
        PORTF &= ~((1<<6)|(1<<5));
    }
    else if(cycle_time < 7000)
    {
        // 역회전
        PORTF &= ~((1<<6)|(1<<5));
        PORTF |= (1<<5);
    }
    else
    {
        // 정지 1초
        PORTF &= ~((1<<6)|(1<<5));
    }
}
break;

case STATE_RINSE:

    OCR3C = 180;

    if(msec_count - motor_dir_time >= 2000)
    {
        motor_dir_time = msec_count;

        motor_dir ^= 1;
    }
}
break;

```

```

        PORTF &= ~((1<<6)|(1<<5));

        if(motor_dir)
            PORTF |= (1<<6);
        else
            PORTF |= (1<<5);
    }

    break;

case STATE_SPIN:

    OCR3C = 255;

    PORTF &= ~((1<<6)|(1<<5));
    PORTF |= (1<<6);

    break;

default:
    break;
}
}

static void washer_led_display(void)
{
    switch(state)
    {
        case STATE_WASH:
            PORTA = 0x01;    // LED0
            break;

        case STATE_RINSE:
            PORTA = 0x02;    // LED1
            break;

        case STATE_SPIN:
            PORTA = 0x04;    // LED2
            break;

        case SET_WASH_TIME:
            PORTA = 0x08;    // LED3

```

```

        break;

        case SET_RINSE_TIME:
            PORTA = 0x10;    // LED4
            break;

        case SET_SPIN_TIME:
            PORTA = 0x20;    // LED5
            break;

        case STATE_PAUSE:
            PORTA = 0x80;    // LED7
            break;

        default:
            PORTA = 0x00;
            break;
    }
}

//
//static void washer_debug(void)
//{
//    //static uint32_t old_ms = 0;
//
//    //
//    //if(msec_count - old_ms >= 1000)
//    //{
//        //old_ms = msec_count;
//
//        //printf("STATE=%s ", state_string());
//        //printf("TIME=%d\r\n", remain_time);
//        //
//    //}
//}

void washer_run(void)
{
    switch(state)
    {
        case SET_WASH_TIME:
            fnd_display_setting(wash_time);

```

```

        break;

        case SET_RINSE_TIME:
            fnd_display_setting(rinse_time);
            break;

        case SET_SPIN_TIME:
            fnd_display_setting(spin_time);
            break;

        default:
            fnd_display_washer(remain_time, remain_msec);
            break;
    }

    washer_button_process();

    washer_timer_update();

    washer_state_process();

    washer_motor_control();

    washer_led_display();

    //washer_debug();
}

```

```

#washer.h
/*

```

```
* washer.h
*
* Created: 2026-06-18 오후 3:59:06
* Author: kccistc
*/
```

```
#ifndef WASHER_H_
#define WASHER_H_
```

```
#include <stdint.h>
```

```
typedef enum
{
    WASHER_IDLE,

    SET_WASH_TIME,
    SET_RINSE_TIME,
    SET_SPIN_TIME,

    STATE_WASH,
    STATE_RINSE,
    STATE_SPIN,

    STATE_PAUSE

```

```
} WasherState;
```

```
#endif
```